Deadline: *Sunday 8. November 23:59* on Canvas.

Group size: This assignment can be solved in groups of two or three. You are allowed to complete the assignment on your own if you want, but we advise you to solve it in groups as we believe that group discussions are important. Via Canvas you should also be able to see if there are groups with open spots that you could contact. It is okay to discuss the assignment with other groups, but each group should write an individual report.

Group signup: You find the group signup under `People` → `Groups` → `GA2 group` in the menu on the left on Canvas. You and your group should sign up in the same group.

Hand in: Hand in the code as **the zip created by running *create_handin.py*** from the handout folder, with `--group` set to your group number. Do not use any libraries that are not part of the TTK4250 environment. Hand in the report as a single pdf. Only one student per group should hand in both the report and the code.

Code content:

The assignment is pass/fail. The number of tests your code passes is part of how we evaluate the submission as a whole, together with the report. If you notice the tests having false negatives or positives, or behaving weirdly, please let us know.

Report content:

For the report you are limited to 1300 words and a total of 7 pages. Figure captions do not count toward the limit, as long as they are concise and purely descriptive. The group number and group members should be clearly stated.

In the report we require you to clearly answer Task 2, 3 and 4. As the report is short you can skip the introduction and conclusion. You can assume the person reading the report has good knowledge of the subject. You can simply refer to equations and algorithms in the book when what you want to say is already stated there. Ex: "Given what's stated in (12.34) and equation (56.78) in the book the oscillations in Figure 9 are expected" (When referring to equations, figures etc. you should make it clear if you are referring to equations, figures etc. in the book or report).

The goal of the report is to show that you understand the code and the theory behind it by presenting and discussing the plots and results. Remember to argue for your claims and explain your reasoning: a statement that is right but not explained does not show understanding. Ex: If you write "These parameters gave bad results", you should also include something like "as can be seen by the high RMSE values in Figure 9".

It is not expected that you present all the results, plots, and tuning variables from the code, but rather that you present and discuss the ones you find most relevant for each part of the assignment. The report should, however, include all four main types of plots (state, NIS, NEES, error).

In Tasks 2 and 3 you will experiment with different sets of tuning values. Every run saves the figures below in its run directory (re-plot a finished run with `plot_run`), and prints a summary with the RMSE, the averaged NEES and NIS with their 95% intervals and, on the simulated data, the map-quality counts. You do not need every figure for every set: use the ones that support your discussion, and combine them into multi-panel figures where that saves space.

From the simulated data:

- `final_snapshot`: the trajectory and the map against the ground truth, with the map-quality counts in the title.
- `pose_diagnostics`: the x, y and heading errors with their ± 1, 2 and 3σ envelopes and RMSE.
- `pose_nees`: the pose NEES over time with its 95% bounds and the share inside them, and the ANEES with its interval.
- `landmark_nis`: the landmark NIS per degree of freedom over time with its 95% bounds and the share inside them, and the ANIS with its interval.
- `wrong_association` (Task 4 only): the landmark NIS and the association counts around the forced step.

From Victoria Park, which has no ground truth:

- `final_snapshot`: the trajectory and the map against the GPS.
- `position_nis`: the NIS of the GPS positions, shown like the landmark NIS.

- `landmark_nis`: as for the simulated data.
- `landmark_growth`: the number of landmarks over time, which shows when data association starts to fail.

The timing figures (`timing_*` and `landmarks_and_timing`) are there if run time helps your discussion.

We expect you to discuss what these plots and numbers mean in practice, and how they are connected to the different tuning parameters.

Answers and analysis should connect theory and results to the real world, and show your understanding of the problem and the solution. Theoretical or practical insight from the lectures or book elaborated upon or applied to the datasets counts positive.

Other plots, numbers, and considerations are welcome too, but what to show is *up to you* to decide. Try to connect the results on the simulated data to the results of the real data where it is possible.

If you have any questions please make a post on the GA2 discussion thread on Canvas and we will try to answer them as soon as possible or contact the student assistants during the exercise class.

Some background on the template code

The template code is a trimmed-down version of a factor graph SLAM system built on GTSAM, the library referred to in Sec. 9.5 of the book. GTSAM provides the back-end: it holds the factor graph, maintains the Bayes tree, solves for the MAP estimate incrementally with iSAM2, and recovers marginal covariances from it. Everything around that is the front-end: turning odometry and lidar data into factors, predicting the measurements and their innovation covariance, deciding which measurement belongs to which landmark, and when a new landmark is born. Data association itself (JCBB) is given; the factors and the quantities it relies on are yours to implement in Task 1.

Set up the environment from the handout folder with

```
conda env create -f environment.yml
conda activate ttk4250_ga2
pip install -e . --no-deps
```

Run the tests with `pytest`. Runs are started with `run_sim` and `run_real`, and an existing run can be re-plotted with `plot_run <run directory>`. Ctrl+C stops a run and still saves it up to the last completed step; add `--no-save-on-abort` if you would rather it just exit. Setup problems, especially on Windows, are covered in `docs/INSTALL.md`.

The files handed out are:

- `configs/`: the run configurations. Everything marked `# TODO tune` is yours to experiment with in Tasks 2 and 3.
- `docs/GTSAM.md`: every GTSAM call Task 1 needs, with what each one means. Read it before you start.
- `preprocessing.py`: the odometry and lidar front-end. `relative_pose` and `preintegrate` are yours; the lidar part at the bottom of the file, which turns a raw scan into range-bearing measurements, is given.
- `factor_graph.py`: pose prediction, the odometry and landmark factors, the measurement model and its inverse, the joint covariance assembled from GTSAM's marginals, and the innovation covariance. Most of your work is here.
- `landmark_manager.py`: landmark birth. The M of N rule is yours; the rest is given.
- `data_association.py`: the JCBB algorithm, and the forced wrong association for Task 4. **NB:** you do not need to do anything in this file.
- `evaluation.py`: the NEES, the NIS, their bounds and the run summary, and, on the simulated data, the comparison of the finished map with the true landmarks. **NB:** you do not need to do anything in this file.
- `slam.py`: the main loop that ties all of the above together. Reading it is the fastest way to understand what happens in one time step. **NB:** you do not need to do anything in this file.
- `loaders/`, `logger.py`, `plotter.py`, `plotting/`: dataset adapters, run logging, and all the plotting. **NB:** you do not need to do anything in these files.

Task 1: *Implement factor graph SLAM*

A factor graph SLAM system turns sensor data into a graph and solves it. The back-end (iSAM2, through GTSAM) does the solving: it optimizes the graph and recovers covariances from it. Everything around it is yours to implement in this task: turning wheel odometry into relative poses (a, b), adding the factors (c), predicting each measurement and how uncertain that prediction is (d–f), which is what the given JCBB needs to decide which measurement belongs to which landmark, and placing and confirming new landmarks (g, h).

All functions are marked with `# TODO` in the code. The GTSAM calls you need are listed in `docs/GTSAM.md`, and your editor shows their signatures.

Testing your code. Each part has its own tests, selected by a pytest marker named after the part: `pytest -m b` runs the tests for (b), and `pytest -m c2` those for (c2). Add `-x` to stop at the first failure. Use them to check each part as you go. `pytest` runs everything, including the end-to-end tests, which need all parts.

(a) Implement `preprocessing.relative_pose`, which turns one wheel-encoder reading into a relative pose increment.

The Victoria Park vehicle reports a forward speed v_e measured at the left rear wheel and a steering angle α. The encoder sits a distance H off the centre line, so during a turn it travels along a different arc than the centre of the rear axle. With L the wheelbase, the distance between the axles, the kinematic bicycle model gives the body twist

$$v_b = \frac{v_e}{1 - \frac{H}{L}\tan\alpha}, \qquad \omega = \frac{v_b}{L}\tan\alpha, \qquad \xi = \begin{bmatrix} v_b & 0 & \omega \end{bmatrix}^T.$$

Held constant over the time Δt between two odometry readings, this twist integrates *exactly* to $\Delta T = \mathrm{Exp}(\xi\,\Delta t)$, which follows a circular arc.

> *Hint:* The exponential map is covered in Sec. 6.2.3. `gtsam.Pose2.Expmap` implements it, taking $\xi\Delta t$ as a numpy array. The vehicle geometry is the `car` argument: H is `car.H` and L is `car.L`.

(b) Implement `preprocessing.preintegrate`, which compounds a sequence of relative poses into a single one, with corresponding covariance.

Wheel odometry arrives far faster than lidar scans. Rather than putting a pose in the graph for every encoder reading, all increments between two scans are compounded into one relative pose and one covariance, which is added to the graph as a single `BetweenFactorPose2`. This is called *preintegration* (Sec. 9.1).

The mean is repeated composition. For the covariance, propagate it through the Jacobians of each composition:

$$\Sigma \leftarrow H_1 \Sigma H_1^T + H_2 \Sigma_i H_2^T,$$

where Σ_i is the covariance of increment i, and H_1 and H_2 are the Jacobians of `compose` with respect to its first and second argument.

> *Hint:* Start the recursion from the identity pose `gtsam.Pose2()` and a zero 3×3 covariance. `compose` fills in both Jacobians for you if you pass it two arrays preallocated as `np.zeros((3, 3), order="F")`.

(c) Build the graph. Each factor you add is one term of the negative log posterior (9.4).

(c1) Implement `factor_graph.predict_pose`, which returns the predicted pose

$$\hat{x}_k = \hat{x}_{k-1} \oplus u_k,$$

the previous pose estimate compounded with the odometry increment u_k, which is expressed in the body frame of x_{k-1}. For a `gtsam.Pose2`, $\oplus$ is `compose`.

Unlike the prediction step of an EKF, this prediction does not alter the estimate. It only gives the optimizer a starting guess for the new pose x_k, which an iterative solver needs for every variable. What connects x_k to x_{k-1} is the odometry factor you add in (c2).

(c2) Implement `factor_graph.add_odometry_factor`, adding the middle term of (9.6) as a `BetweenFactorPose2`.

Hint: The factor is constructed as

```
gtsam.BetweenFactorPose2(key_previous, key_current, measured, noise_model)
```

and `graph.add(factor)` adds it to the graph. Use `gtsam.noiseModel.Gaussian.Covariance`, which takes the full 3×3 matrix, and not `.Sigmas`, which takes only standard deviations. The covariance you preintegrated in (b) is not diagonal. A heading error also moves the pose sideways, so the two are correlated. Dropping that correlation makes the factor too confident in some directions and not confident enough in others.

(c3) Implement `factor_graph.add_landmark_factor`, adding the last term of (9.6) as a `BearingRangeFactor2D`.

Hint: The factor is constructed as

```
gtsam.BearingRangeFactor2D(pose_key, landmark_key, bearing, range, noise_model)
```

with the bearing *first*, as a `gtsam.Rot2(angle)`, and the range as a float. A plain float bearing raises a `TypeError`. The noise model you are handed is already ordered [bearing, range] to match.

(d) Implement `factor_graph.predict_measurement`, the measurement function $\mathbf{h}(x_k, m^i)$ of (9.6), together with the two blocks of the Jacobian $\mathbf{J}$ in (9.8) belonging to this pose and this landmark. It returns the predicted measurement $z = [\text{range}, \text{bearing}]$, the pose block $\partial z / \partial x_k$ (2×3) and the landmark block $\partial z / \partial m^i$ (2×2). You do not derive any of this yourself. GTSAM computes the values and the Jacobians, one row at a time. Your task is to assemble them:

1. Preallocate four sub-Jacobians, one row each: range and bearing, each with respect to the pose (1×3) and to the landmark (1×2), e.g. `H_range_pose = np.zeros((1, 3), order="F")`. Any other shape or type makes GTSAM raise a `TypeError`.
2. Pass them to the two GTSAM calls. Each returns its value and fills in its two sub-Jacobians:

   ```
   pose.range(landmark, H_range_pose, H_range_landmark)
   pose.bearing(landmark, H_bearing_pose, H_bearing_landmark)
   ```

   `bearing` returns a `gtsam.Rot2`, not a float; take `.theta()` for the angle.
3. Stack the rows with `np.vstack`, range first and bearing second, to match the order of z.

(e) Implement `factor_graph.assemble_joint_covariance`, which builds the joint covariance P of the current pose and the landmarks in the local map: the P in $S = HPH^T + R$ of (f).

A factor graph stores the sparse information matrix Λ, not the covariance. GTSAM computes P from Λ^{-1} for the variables you ask for (Sec. 9.4). It hands you P as a `gtsam.JointMarginal`, in blocks:

```
joint_marginal.at(key_i, key_j)
```

is the covariance between the two variables `key_i` and `key_j`. Your task is to put these blocks together into one matrix, in the order of `keys`: the pose first, then the landmarks. Either stack the blocks with `np.block`, or preallocate P and fill it block by block in two loops over `keys`. For the loops, the size of each variable's block is `joint_marginal.at(key, key).shape[0]`.

Be sure to include every block, also the ones pairing two different landmarks, $P_{m^i m^j}$. Leaving them out gives a plausible-looking but wrong S. Also keep the order of `keys`, since (f) stacks the Jacobians in that order.

(f) Implement `factor_graph.innovation_covariance`, which assembles

$$S = HPH^T + R$$

for the whole local map at once. This is (9.26) with P the joint covariance assembled in part (e), H the stacked Jacobian $\mathbf{J}_a$ from part (d), and R the block-diagonal measurement noise $\mathbf{R}_a$.

Here $n =$ `len(jacobians_pose)` is the number of landmarks in the local map. Keep the dimensions in mind: P is $(3+2n) \times (3+2n)$, pose first and then each landmark; H is $2n \times (3+2n)$, two rows per predicted measurement;

R and the returned S are $2n \times 2n$. For $n = 3$:

$$H = \begin{array}{c} \\ z_0 \\ z_1 \\ z_2 \end{array} \begin{array}{c} \begin{array}{cccc} x_k & m_0 & m_1 & m_2 \end{array} \\ \begin{pmatrix} H_0^x & H_0^m & 0 & 0 \\ H_1^x & 0 & H_1^m & 0 \\ H_2^x & 0 & 0 & H_2^m \end{pmatrix} \end{array}$$

where H_i^x is `jacobians_pose[i]` (2×3) and H_i^m is `jacobians_landmark[i]` (2×2). H is dense in its three leftmost columns, because every predicted measurement depends on the pose, and block diagonal elsewhere, because predicted measurement i depends only on landmark i. R is `measurement_cov` repeated n times along its diagonal, since the measurements are independent.

(g) Implement `factor_graph.inverse_measurement`, the inverse measurement function $\mathbf{h}^{-1}(z, x)$: given a measurement z taken from pose x, it returns the landmark's position in the world frame. It places a new landmark for a measurement that JCBB could not associate. Such a measurement may also be clutter, so (h) decides when the new landmark enters the graph.

Unlike in EKF-SLAM (Sec. 7.2), only the position is needed, not a covariance: once the landmark is connected by factors, its covariance follows from the graph, like that of every other variable.

> *Hint:* Find the landmark in the body frame first. `rotate` needs a point, not a range, so start from `gtsam.Point2(range, 0.0)`, the landmark straight ahead, and let `gtsam.Rot2(bearing).rotate(...)` turn it to the measured bearing, giving $(r \cos b,\ r \sin b)$. Then `pose.transformFrom(...)` takes that point from the body frame to the world frame.

(h) Implement `landmark_manager.TentativeLandmark.is_confirmed`, the M of N confirmation rule from target tracking: confirm a tentative landmark once it has been seen in at least M of the last N time steps, the current one included.

The evidence is the list `self.supporting_observations`, with one entry per time step the landmark was seen in. The given code fills it, starting with the measurement that created the landmark. Each entry's time step is in `.step`. Your function only reads the list: count the entries whose `.step` lies inside the window, and compare with M.

> *Hint:* Both ends are easy to get off by one. With $N = 3$ and `current_step` 10, the window is steps 8, 9 and 10: it starts at $10 - 3 + 1$, and a step equal to the start counts, so compare with $\geq$. Exactly M hits is enough, so compare the count with $\geq$ as well.

Task 2: *Run factor graph SLAM on simulated data*

The simulated dataset consists of

- odometry (K, 3): the measured movement of the robot in its body frame,
- z (K,): a list of the detections at each time step, each of shape (number of measurements, 2),
- poseGT (K, 3): the true pose at each time step,
- landmarks (L, 2): the true landmark positions.

The initial tuning is in `configs/sim_default.yaml`. Everything marked `# TODO tune` is yours to change. It is a *working baseline*: the run completes and the map is mostly right, but there is room for improvement in both accuracy and consistency. The pose NEES sits above its expected value, and the landmark NIS below it. With a correct Task 1, the baseline maps 72 of the 78 true landmarks, with 3 spurious ones, and gives a pose ANEES of 8.37. The run is deterministic, so if yours differs, check Task 1 before you start tuning.

Running. From the handout folder, run the baseline with

```
run_sim --output-dir runs/sim/baseline
```

For each tuning set, copy `configs/sim_default.yaml` to a new file in `configs/`, change its values, and run it with `--config` pointing at that file and its own `--output-dir`. A run takes about 10–15 seconds. The run directory keeps

the figures, the config used and the summary (in `metadata.json`). `plot_run <run directory>` re-plots a finished run, and `--no-show-plots` skips the plot windows.

Find 3 sets of tuning values that differ from the baseline in different ways, and explain the results you get with them. For example:

- a set that performs better than the baseline in some way,
- a set with similar errors but poor consistency,
- a set that maps noticeably fewer landmarks, and how that affects the trajectory.

Every run prints a summary of the averaged NEES and NIS (the landmark ANIS per degree of freedom, since the number of associated landmarks varies). To judge consistency, compare each average with its expected value, and check that about 95% of the per-step values fall inside their bounds. Do not rely on the 95% interval of the average alone: (4.67) assumes independent errors, but the errors of neighbouring poses in a smoother are strongly correlated, so even with the correct noise, one run's ANEES can land anywhere between about 1.5 and 8.

Every simulated run also prints a map-quality summary, which compares the map with the true landmarks. Each map landmark is paired with at most one true landmark, closest pairs first and only within 3 m, and only the true landmarks the run actually observed are counted:

- *Mapped*: a true landmark with a map landmark paired to it. The RMS position error is taken over these pairs.
- *Missed*: an observed true landmark with no map landmark paired to it. Either it was never confirmed, because it was not seen in M of N steps or its measurements kept being associated elsewhere, or its estimate drifted more than 3 m away.
- *Duplicate*: an extra map landmark within 3 m of a true landmark that already has its pair. Data association failed to match a measurement to the landmark already in the map, and a new landmark was born from it instead.
- *Spurious*: a map landmark with no true landmark within 3 m. It was born from a wrong measurement or a wrong association, or its estimate has drifted far from where the landmark is.

Duplicates point at the front-end (association and landmark birth); missed and spurious landmarks can come from either end.

Note: The runtime of JCBB grows quickly with the number of duplicate landmarks, because each one gives a measurement more candidates to pair with. A tuning that creates duplicates (noise values so small that the gates reject true re-observations, or landmark birth on a single sighting, $M = N = 1$, with a wrong noise model) can slow a run from seconds to minutes. If a run slows down substantially, stop it with Ctrl+C. The run so far will be saved, and its map-quality summary shows whether duplicates are the cause.

Some questions to help discussion (not mandatory to answer):

- Which types of errors do you see, and why do you think they look like this?
- How much of your error is the front-end responsible for, and how much the back-end (Sec. 9.1)? Use the map-quality numbers.
- Would you say that there are any shortcomings of the algorithm on this dataset?

Task 3: *Run factor graph SLAM on the Victoria Park dataset*

Victoria Park is a real dataset: a vehicle with a laser scanner, which sees the tree trunks, drives through a park, with a GPS for reference (see the dataset's web page[1]). The loader converts all times to seconds and turns the raw data into odometry and tree detections for you. The raw data consists of

- LASER (mK, 361): laser scans, 361 returns spread over a half circle,
- speed (K,) and steering (K,): the speed from a wheel encoder at the rear left wheel, and the steering angle,
- Lo_m and La_m (Kgps,): the GPS position in metres,
- timeLsr, timeOdo and timeGps: the time stamps of the scans, the odometry and the GPS, in milliseconds.

The initial tuning is in `configs/real_default.yaml`, and it already works well: the map does not fill up with duplicate landmarks, and the trajectory stays close to the GPS. Consistency, however, can be improved. The landmark NIS sits well below its expected value. Improving this is the main point of this task.

As in Task 2, find 3 sets of tuning values that differ in different ways, and explain the results you get with them. Do not expect to get much closer to the GPS than the baseline: a better set is one that improves consistency without giving up what the baseline does well. At least one set should lower the odometry noise. Some sets will break the run: the map fills up with duplicate landmarks and the trajectory drifts away from the GPS. When that happens, describe what breaks and try to explain why it happens.

Plan your runs. Use `run_real` the way you used `run_sim` in Task 2. A full run over all 7247 laser scans, the default, took about 2.5 minutes on our machine with the given tuning, and about twice that when the map grows; expect it to take longer on a laptop. To find good values without hours of waiting:

- Explore with `run_real --steps 2000`, which takes about 20 seconds. The first 2000 scans cover most of the park, including the first *loop closures*: places where the car returns to trees it has mapped before, so the graph connects its current pose to landmarks seen earlier. A tuning that breaks in this first period will not recover, but one that looks good can still fall apart later. Therefore, be sure to confirm a promising set with a full run.
- Change one group of parameters at a time (odometry noise, measurement noise, gates, landmark birth), so that you know what caused what.
- Watch the number of landmarks printed at the end of each run. The given tuning ends the first 2000 scans with about 140. If several hundred are observed, this likely means that data association is failing.

All figures and numbers in your report should come from full runs.

There is no ground truth, so the plots compare with the GPS instead. Keep in mind that the GPS is far from perfect, and that it drops out for about 40% of the drive, so the GPS NIS and the distance to the GPS only cover the times it is available.

The GPS is not used by the SLAM system, as it only serves as a reference. The GPS NIS compares the estimate with each GPS sample, assuming the GPS has standard deviation `noise.sigma_gnss` (1 m by default). This value is a guess about the GPS. Changing it does not change the estimate, only the GPS NIS. A larger value gives a lower GPS NIS for the same estimate. Do not tune it to improve the plot.

Some questions to help discussion (not mandatory to answer):

- Which types of errors do you see, and how do they differ from those on the simulated data? Why?
- What happens to the estimate of the whole trajectory when a loop is closed?
- What shortcomings does the algorithm show on this dataset, and how could it be improved?

[1] http://www-personal.acfr.usyd.edu.au/nebot/victoria_park.htm

Task 4: *One bad association*

In this short task, we will look at what happens when a data association is wrong, and how a robust loss function can help. The experiment is quick to run, and its discussion should fit in one figure and a few short paragraphs.

The config `configs/task4.yaml` sets `inject_wrong_at_step: 600` under `association`, which re-pairs one measurement with the wrong landmark at step 600. Everything else is the same as in `configs/sim_default.yaml`, the default tuning from Task 2. Run it with

```
run_sim --config configs/task4.yaml --output-dir runs/sim/task4
```

and compare it with your baseline run from Task 2: explain what this single wrong association does to the trajectory and the map, using the error values and landmark counts from the run summary. Besides the usual figures, this run saves `wrong_association.pdf`, which shows the landmark NIS on a log scale and the number of associated measurements around step 600. How does the landmark NIS behave at step 600 and in the steps after it, and why?

The Huber loss is a robust loss function (GTSAM calls it a robust kernel, hence the config name) that is quadratic for small errors and linear for large ones:

$$\rho_\delta(e) = \begin{cases} e^2, & |e| \le \delta, \\ 2\delta|e| - \delta^2, & |e| > \delta. \end{cases}$$

With `backend.robust_kernel: huber`, each landmark term in (9.6) changes from

$$\|h(x_k, m^i) - z_k^i\|_R^2 \quad \text{to} \quad \rho_\delta\big(\|h(x_k, m^i) - z_k^i\|_R\big),$$

where $\|h(x_k, m^i) - z_k^i\|_R$ is the measurement error normalized by the measurement noise covariance R (its Mahalanobis distance), so the threshold δ, set by `backend.huber_k`, is in units of measurement noise standard deviations. For more background, see the Wikipedia article on the Huber loss[2], which defines it with an extra factor $\frac{1}{2}$.

Edit `configs/task4.yaml`: set `backend.robust_kernel: huber`, and run the simulation again with

```
run_sim --config configs/task4.yaml --output-dir runs/sim/task4_huber
```

Explain how this changes the cost function (9.6) for the wrong measurement, and why that helps; the two runs' `wrong_association` figures show the difference.

Finally, compare with EKF-SLAM (Sec. 7.2): what are your options for recovering from a wrong association in each of the two frameworks?

[2] https://en.wikipedia.org/wiki/Huber_loss